

# Sri Lanka Institute of Information Technology



SE3090 – Software Engineering Frameworks  
Year 3, Semester 1 - 2026

Practical Answer Submission

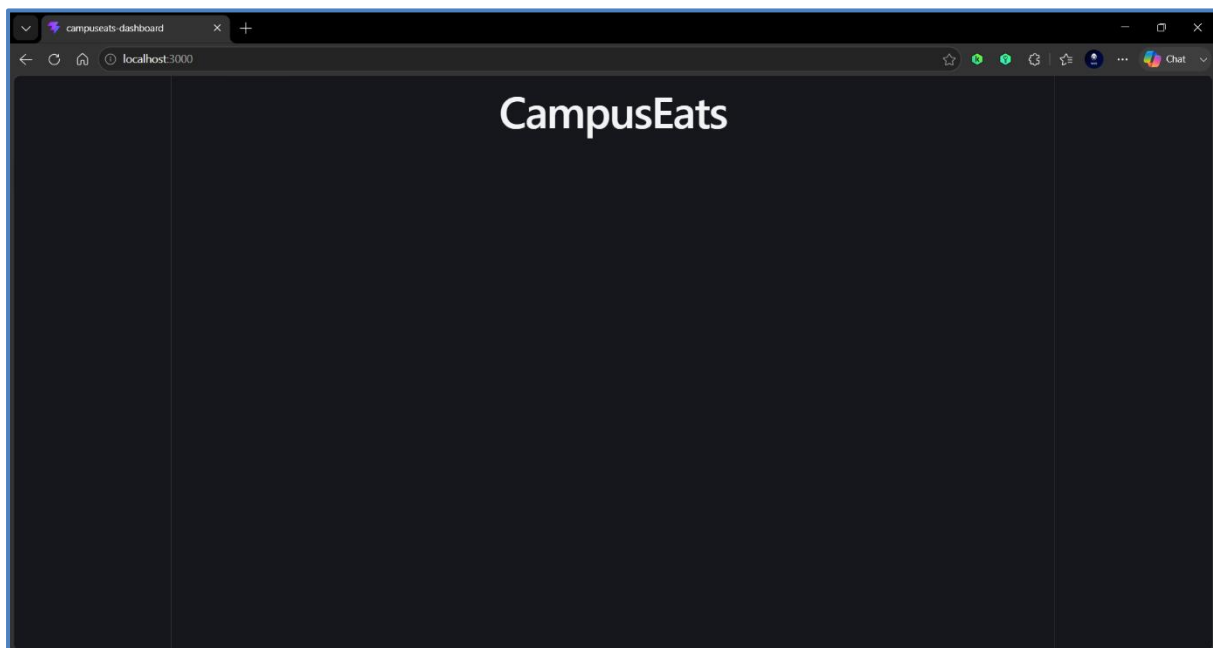
Practical Sheet No: 02

|                     |                           |
|---------------------|---------------------------|
| Student ID          | IT24102798                |
| Student Name        | Sooriyabandara U.R.G.W.K. |
| Campus/ Center name | Malabe                    |
| Specialization      | Artificial Intelligence   |
| Batch No.           | 01.01                     |

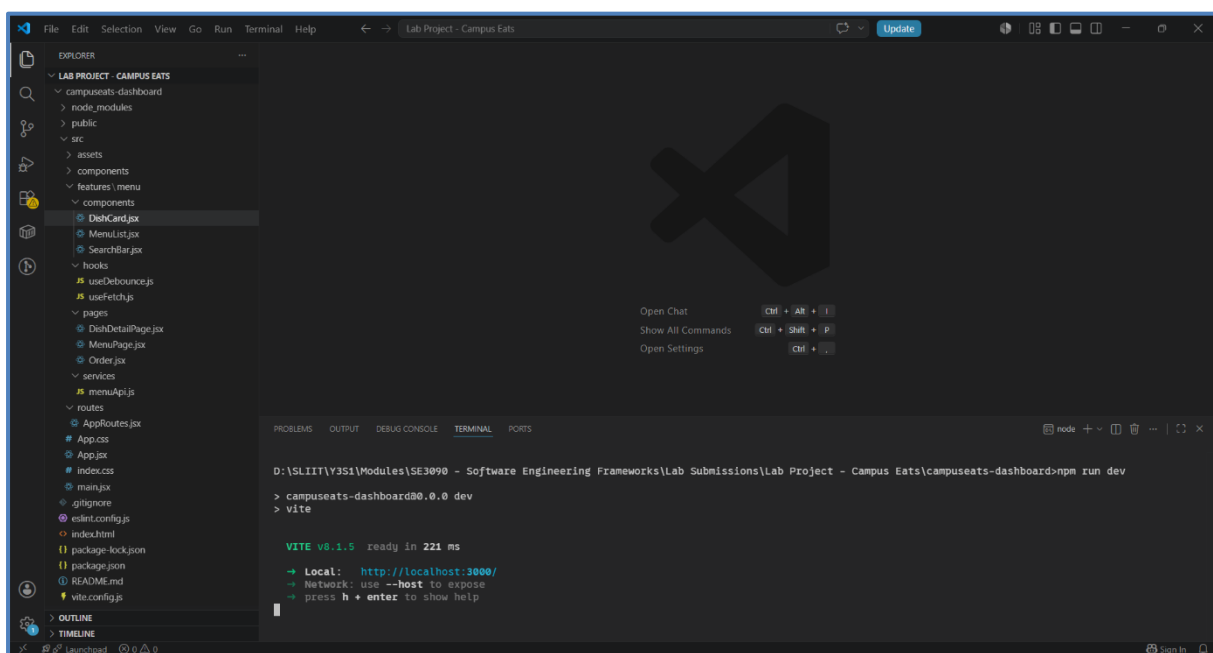
## Task 01 – Scaffold the React App & Feature-Based Structure

### Task Explanation

- This task focused on setting up a new React application using Vite and organizing the project with a feature-based folder structure. Grouping files by feature rather than by file type improves maintainability and scalability and makes the project easier to navigate as it grows.
- Screenshot of the Running App



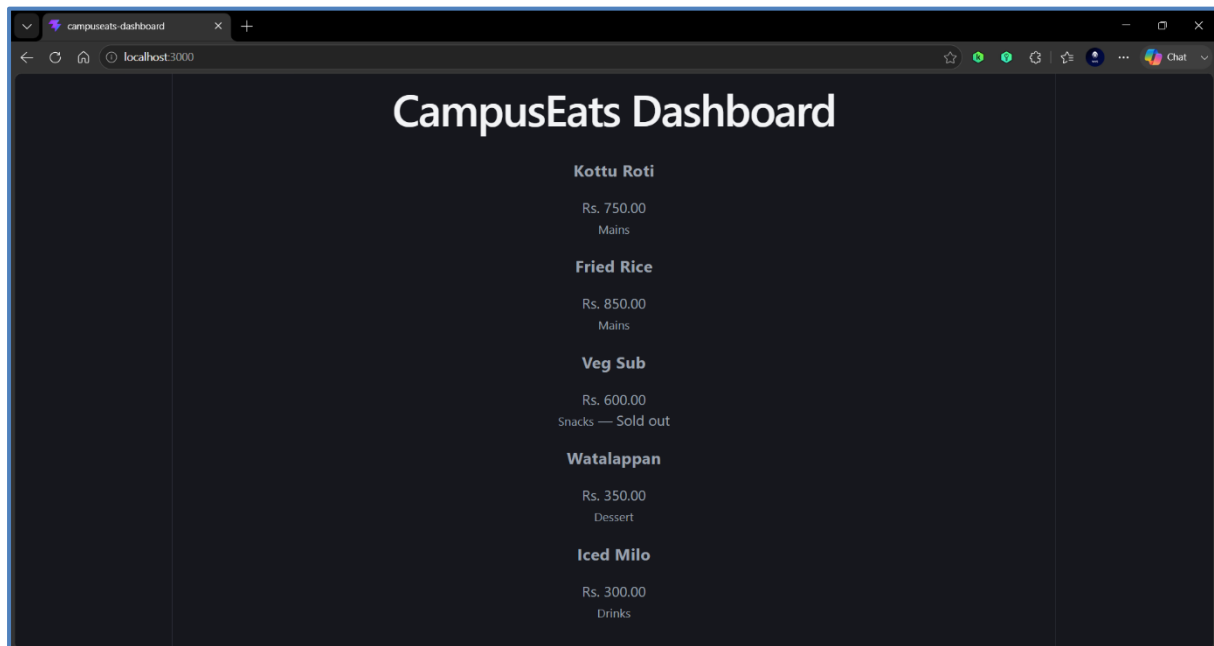
- Screenshot of the Feature-based Folder Structure & the Vite Dev Server



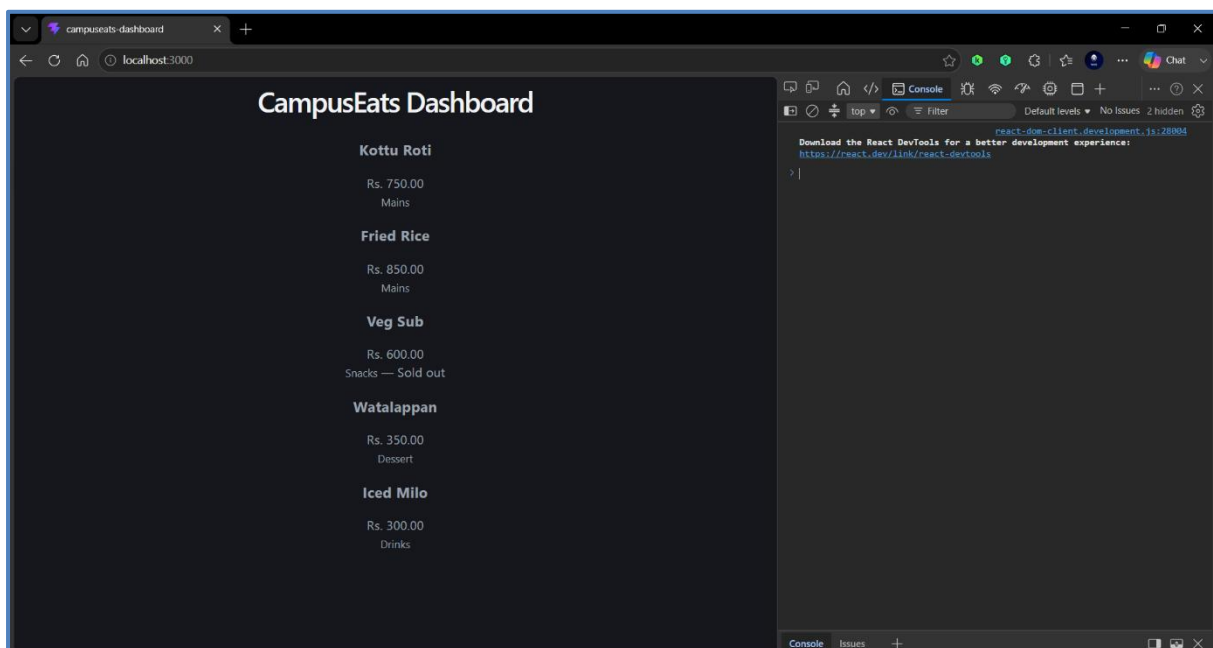
## Task 02 – Reusable Components, Props & List Rendering

### Task Explanation

- This task demonstrated component-based development by creating reusable DishCard and MenuList components. Data was passed from parent to child using props, while list rendering and conditional rendering were used to display menu items efficiently.
- Screenshot of the list of Five Dish Cards rendered from the Array via Props



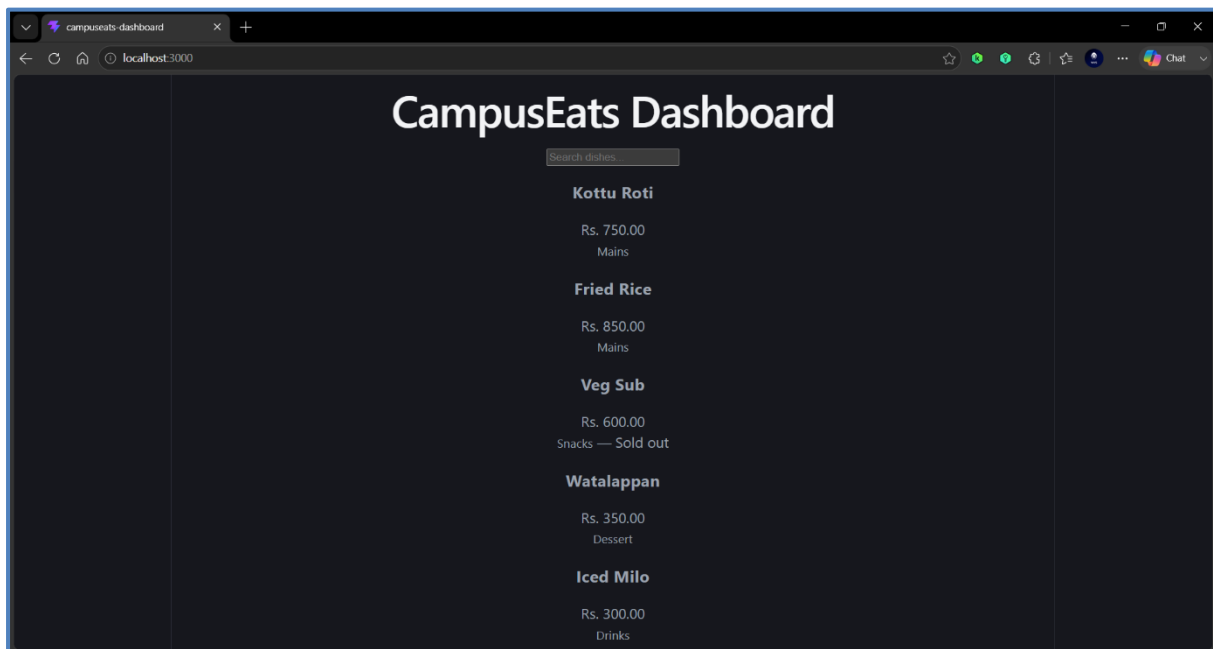
- Screenshot of the Console with No Missing-key Warnings



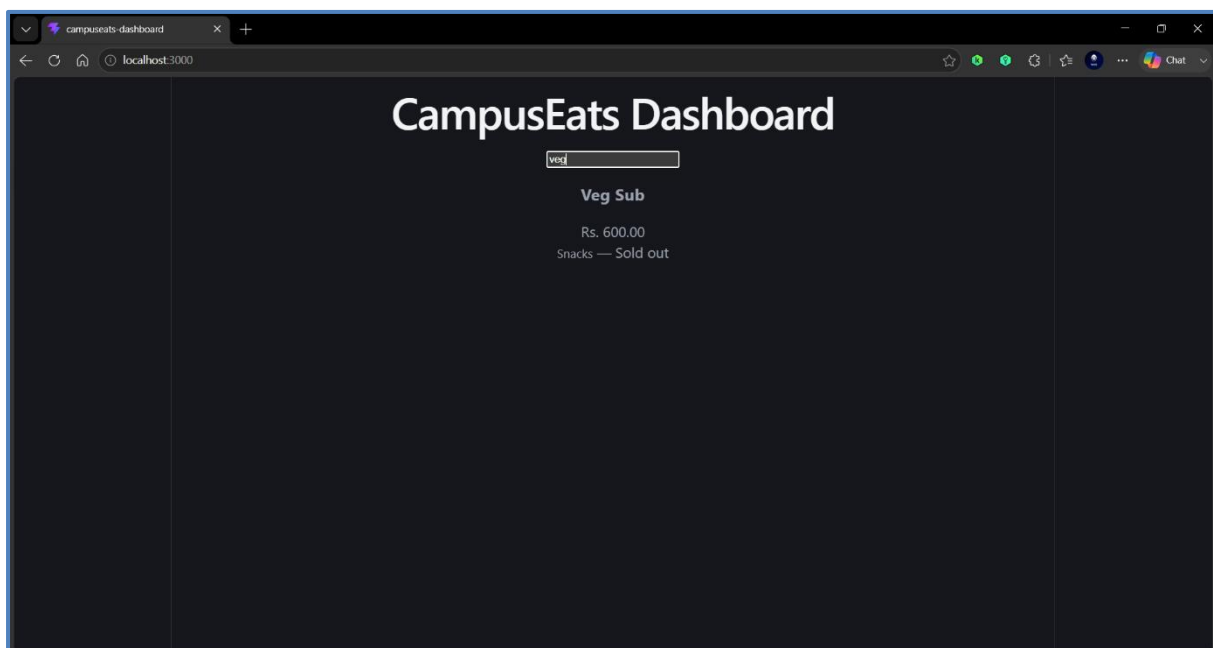
## Task 03 – State, the useDebounce Hook & Event Handling

### Task Explanation

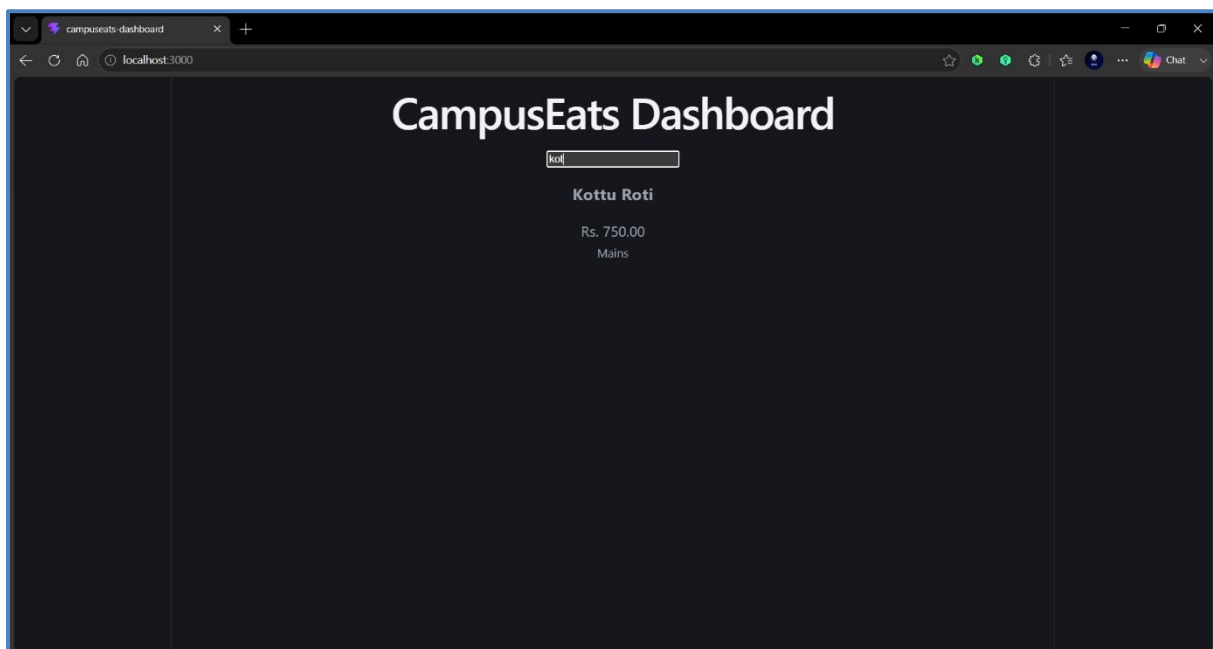
- This task introduced React state management with useState and implemented a controlled search input. A custom useDebounce hook was used to delay filtering until the user stopped typing, reducing unnecessary updates and improving performance.
- Screenshot of the Working Search Box that Filters the Dish List



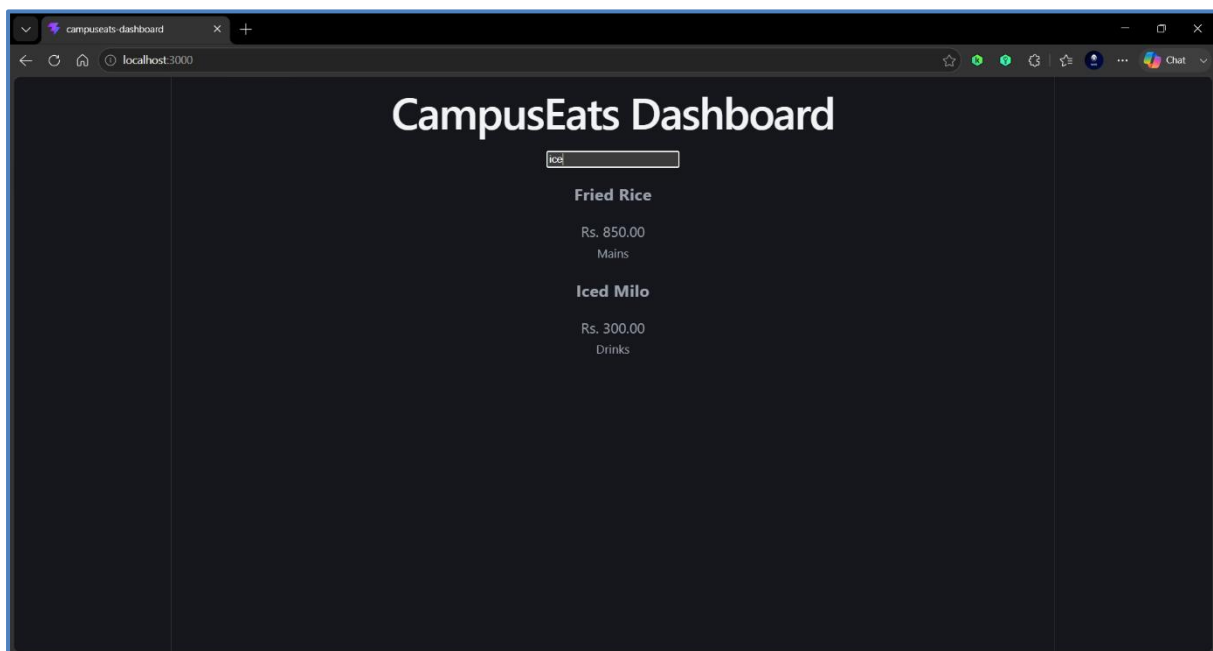
- Screenshot of the Filtered Dish List – 01



- Screenshot of the Filtered Dish List – 02



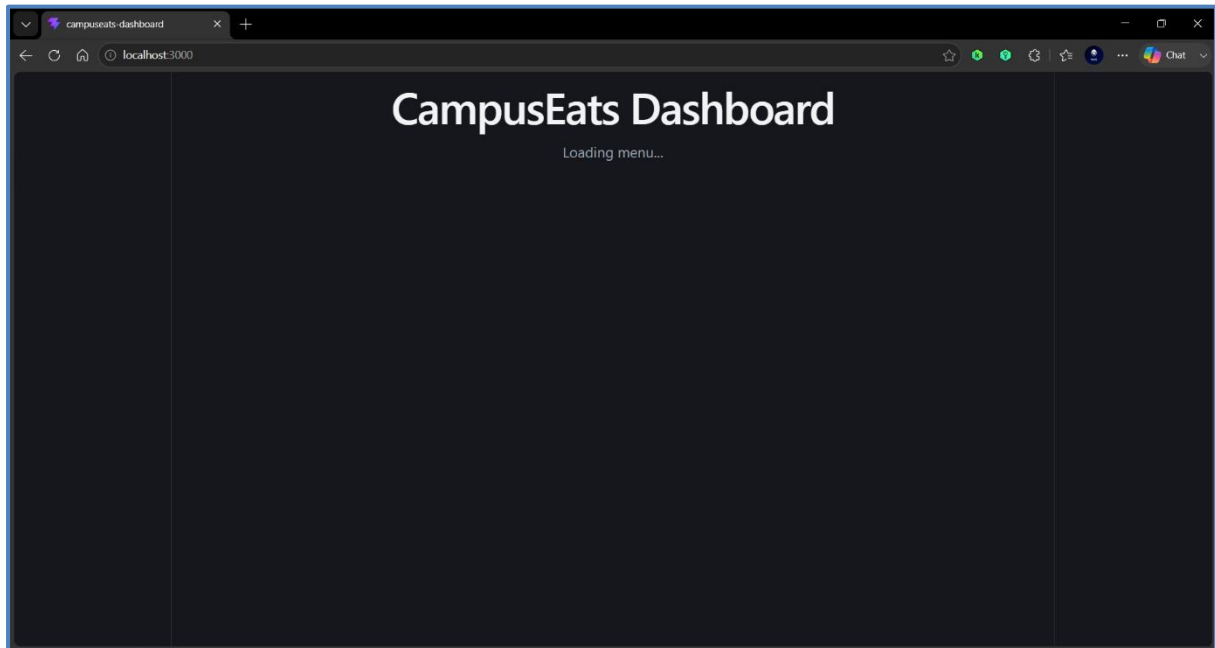
- Screenshot of the Filtered Dish List – 03



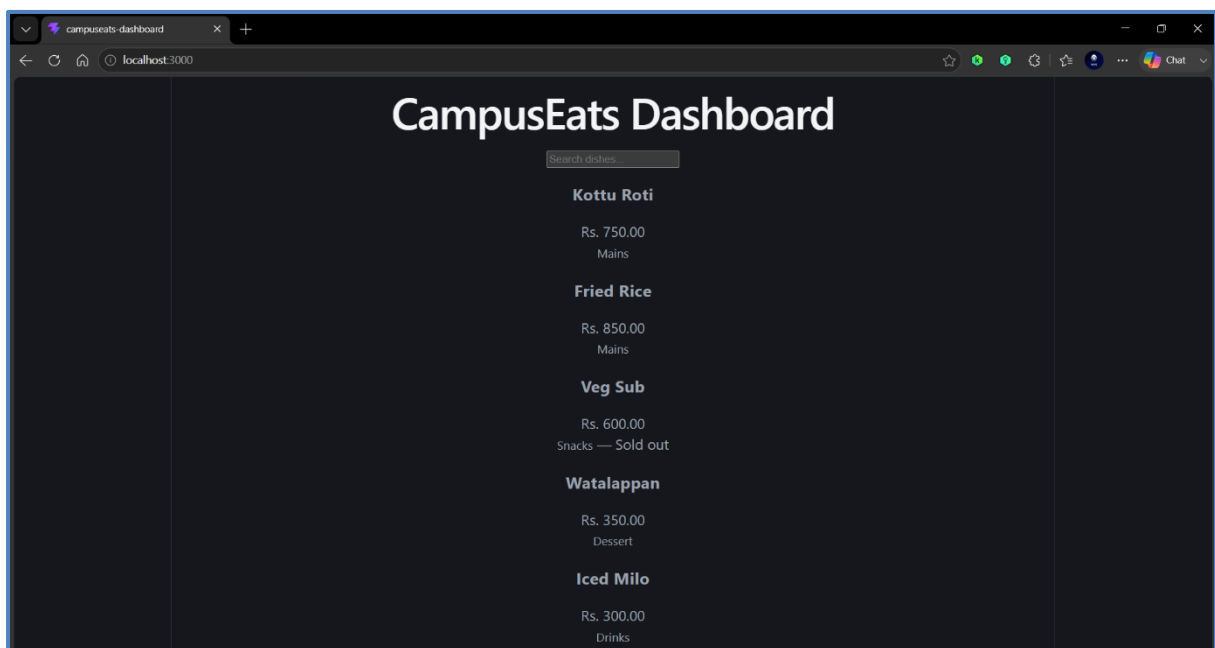
## Task 04 – Data Fetching with Loading, Success & Error States

### Task Explanation

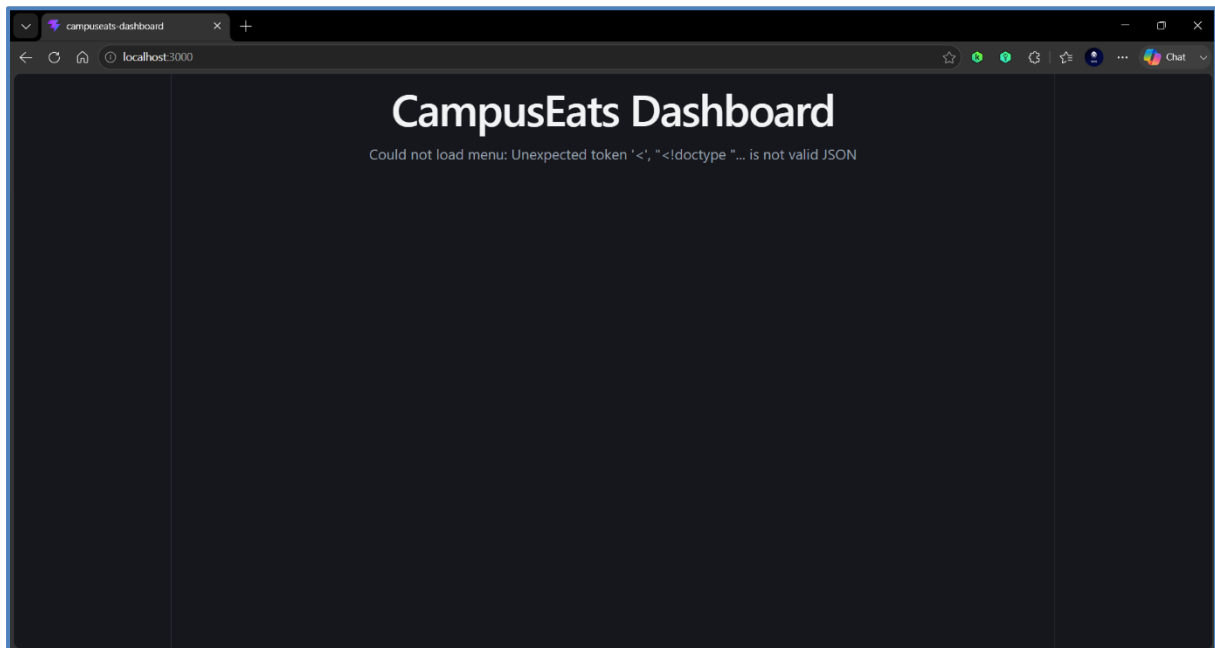
- This task replaced hard-coded data with data fetched from menu.json using a reusable useFetch hook. The application handled the three essential fetch states which are loading, success, and error—to provide a reliable and user-friendly experience.
- Screenshot of the Loading Page



- Screenshot of the Successfully Loaded Page



- **Screenshot of the Error of the Wrongly Fetched JSON**



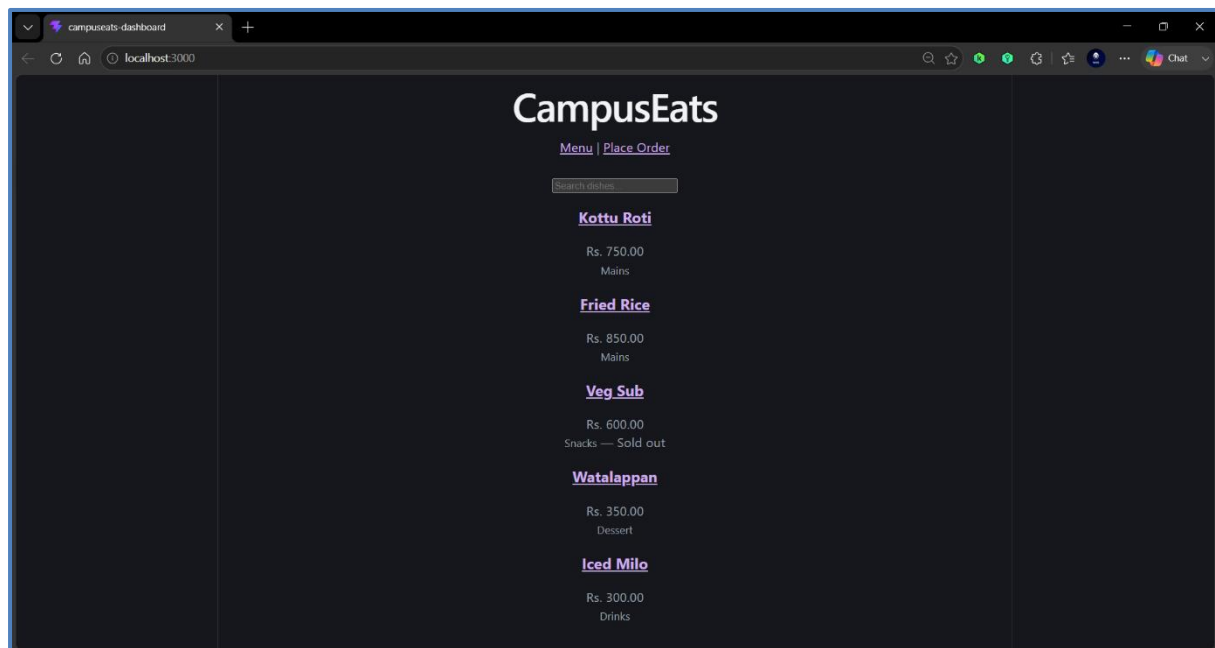
## Task 05 – Routing, Navigation & a Validated Form

### Task Explanation

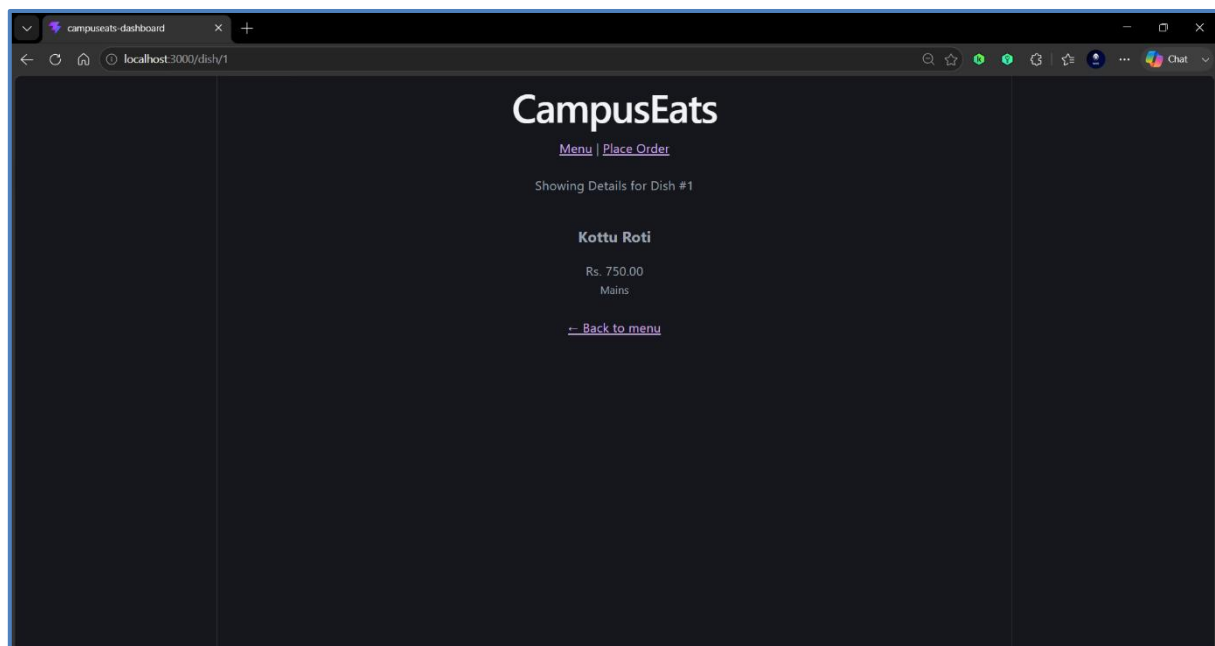
- This task implemented client-side routing with React Router, allowing navigation between the menu, dish details, and order pages while handling invalid routes with a 404 page. It also included form validation using React state to prevent invalid submissions and provide feedback to the user.

### Part A – Routing & Navigation

- Screenshot of the Finalized Routing and Navigation

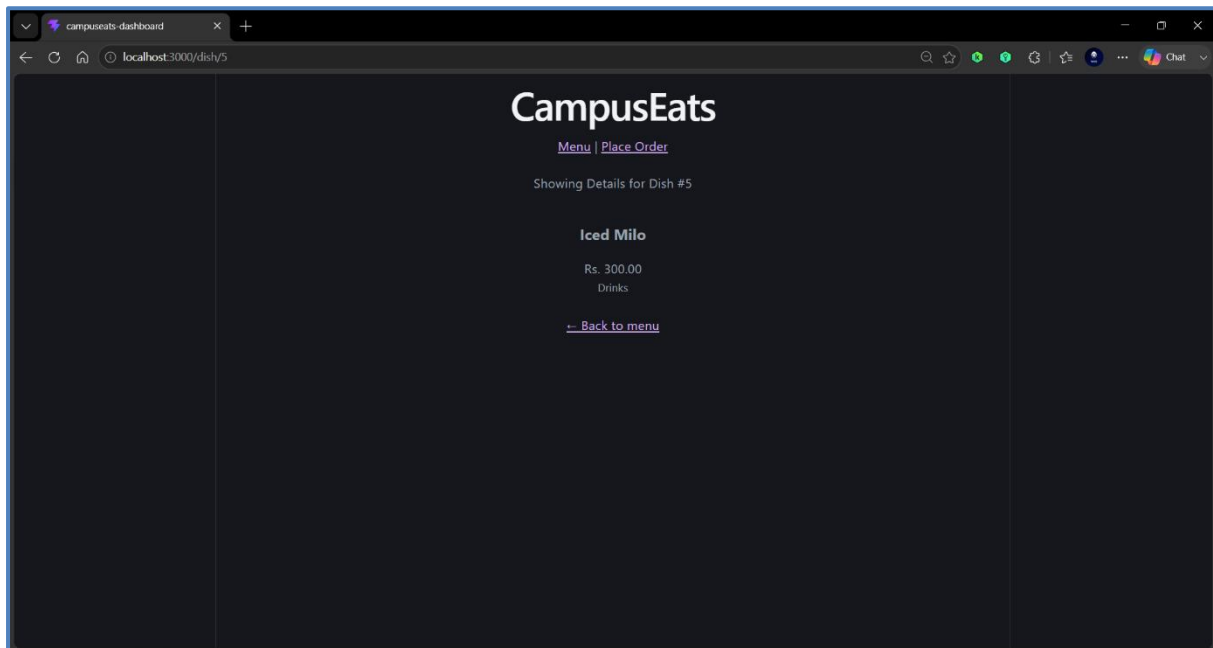


- Screenshot of the Routing of Detailed Pages of Each Dish – 01

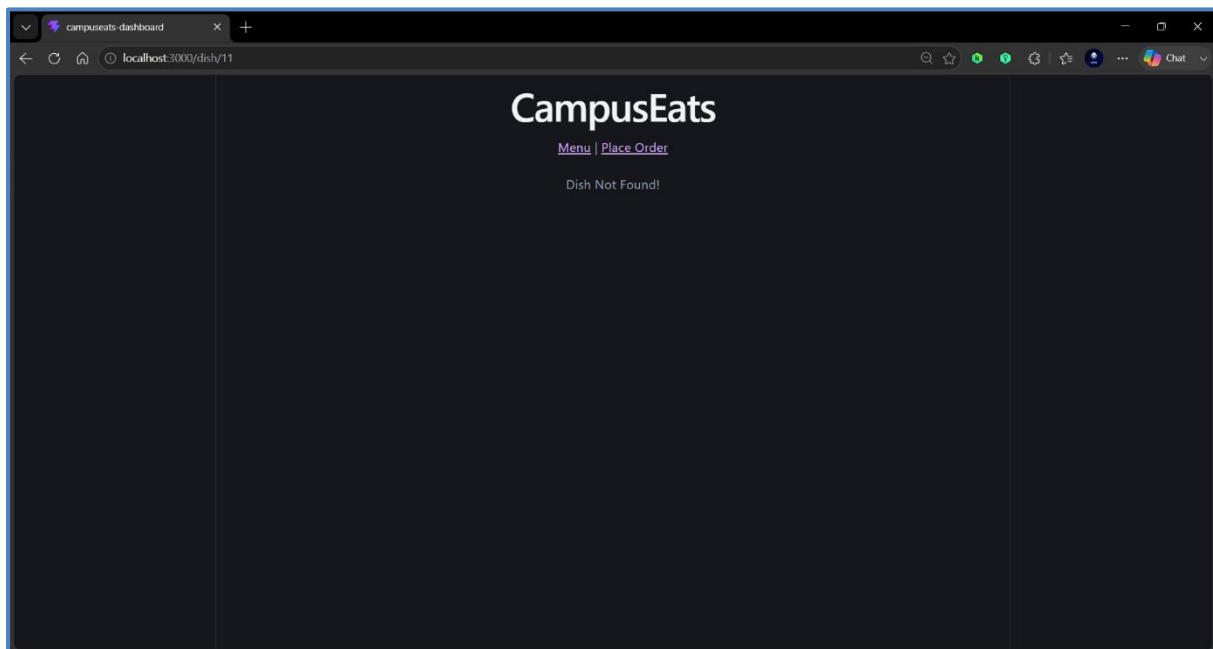




- **Screenshot of the Routing of Detailed Pages of Each Dish – 02**

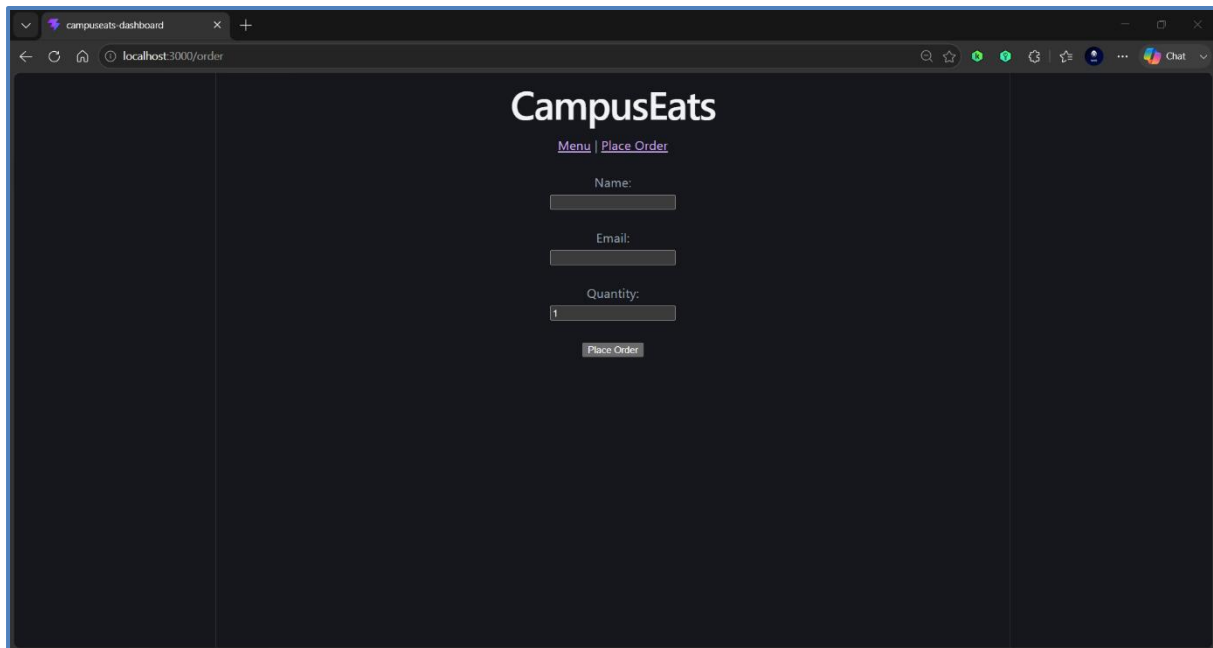


- **Screenshot of the Routing of Detailed Pages of Each Dish – 03**



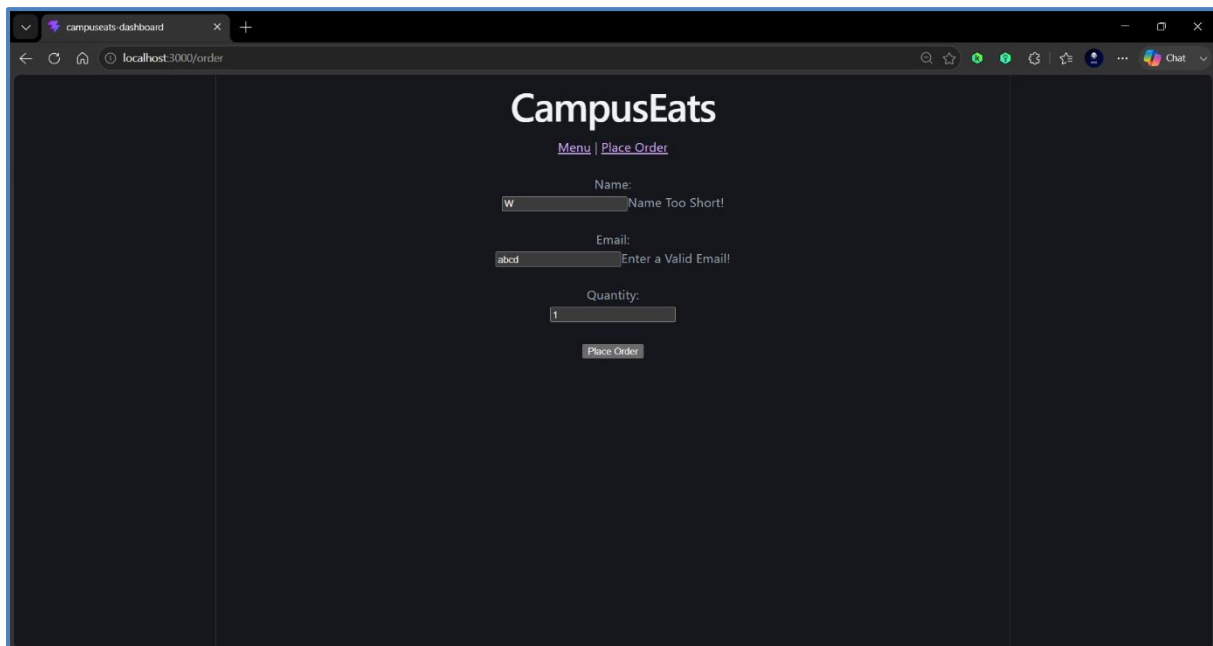
## Part B – A Validated Order Form

- Screenshot of the Order Form



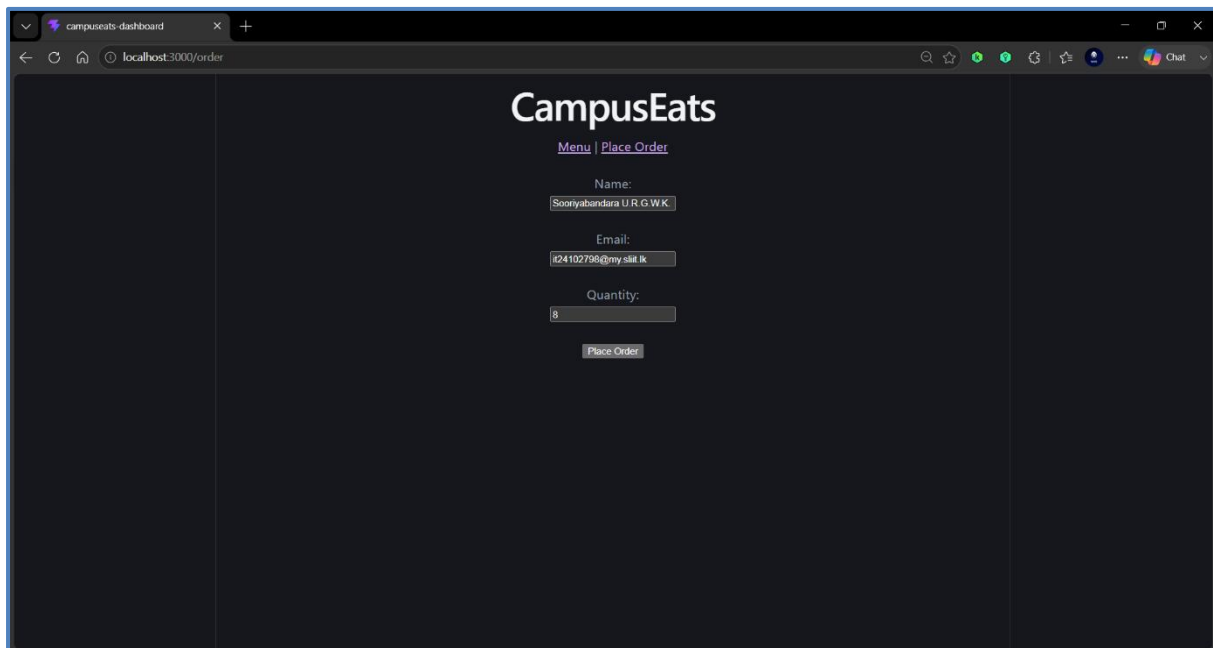
A screenshot of a web browser displaying the 'CampusEats' order form. The browser's address bar shows 'localhost:3000/order'. The page has a dark theme. At the top, the 'CampusEats' logo is centered, with links for 'Menu' and 'Place Order' below it. The form consists of three input fields: 'Name:', 'Email:', and 'Quantity:'. The 'Quantity' field is pre-filled with the number '1'. Below the 'Quantity' field is a 'Place Order' button. The form is centered on the page, and the background is a solid dark color.

- Screenshot of the Incompletely Filled Order Form



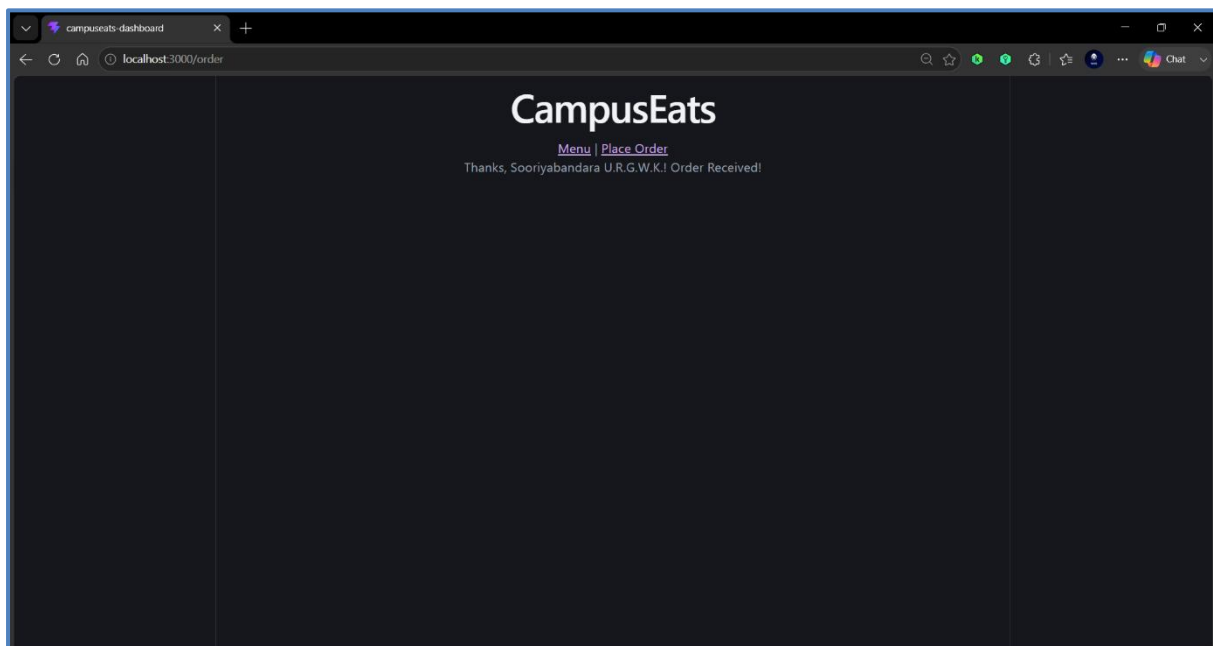
A screenshot of the same 'CampusEats' order form, but with validation errors. The 'Name' field contains the text 'w' and has a red error message 'Name Too Short!' to its right. The 'Email' field contains the text 'abcd' and has a red error message 'Enter a Valid Email!' to its right. The 'Quantity' field is still pre-filled with '1'. The 'Place Order' button remains at the bottom. The browser's address bar still shows 'localhost:3000/order'.

- Screenshot of the Completely Filled Order Form



A screenshot of a web browser showing the 'CampusEats' order form. The browser's address bar displays 'localhost:3000/order'. The form is titled 'CampusEats' and includes links for 'Menu' and 'Place Order'. The form fields are filled with the following information: Name: 'Sooriyabandara U.R.G.W.K.', Email: 'id24102790@gmail.lk', and Quantity: '8'. A 'Place Order' button is visible at the bottom of the form.

- Screenshot of the Successfully Placed Order Form



A screenshot of the same web browser showing the 'CampusEats' order confirmation page. The browser's address bar still displays 'localhost:3000/order'. The page shows the 'CampusEats' logo and links for 'Menu' and 'Place Order'. Below the links, a confirmation message reads: 'Thanks, Sooriyabandara U.R.G.W.K.! Order Received!'. The 'Place Order' button is no longer visible.

**Part C – Justify your state choices (LO4)**

| <b>Piece of State</b>            | <b>Tool you would use</b> | <b>Why (One Line)</b>                                                                     |
|----------------------------------|---------------------------|-------------------------------------------------------------------------------------------|
| Search Box Text                  | useState                  | It is local UI state used only within the MenuPage component.                             |
| Menu Data from the API           | useFetch                  | It manages asynchronous server data while providing loading, success, and error states.   |
| Logged-in User (Shared App-wide) | Context API               | It allows user information to be shared across multiple components without prop drilling. |
| A Cart of Selected Dishes        | Context API               | It stores shared cart data that needs to be accessed by multiple pages and components.    |

## Extra – Source-code Snippets

- DishCard.jsx

```
DishCard.jsx X
campuseats-dashboard > src > features > menu > components > DishCard.jsx > ...
1 import { NavLink } from "react-router-dom";
2
3 export default function DishCard({ dish, showLink = true }) {
4   return (
5     <div className="dish-card">
6       {showLink ? (
7         <NavLink to={`/${dish}/${dish.id}`}>
8           <h3>{dish.name}</h3>
9         </NavLink>
10       ) : (
11         <h3>{dish.name}</h3>
12       )}
13       <p>Rs. {dish.price.toFixed(2)}</p>
14       <small>{dish.category}</small>
15       {dish.available && <span> - Sold out</span>}
16     </div>
17   );
18 }
```

- MenuList.jsx

```
MenuList.jsx X
campuseats-dashboard > src > features > menu > components > MenuList.jsx > ...
1 import DishCard from "./DishCard";
2
3 export default function MenuList({ dishes }) {
4   if (!dishes.length) return <p>No dishes match your search.</p>;
5   return (
6     <div className="menu-grid">
7       {dishes.map((dish) => (
8         <DishCard key={dish.id} dish={dish} />
9       ))}
10     </div>
11   );
12 }
```

- useDebounce.js

```
# useDebounce.js X
campuseats-dashboard > src > features > menu > hooks > # useDebounce.js > ...
1 import { useState, useEffect } from "react";
2
3 export function useDebounce(value, delay = 400) {
4   const [debounced, setDebounced] = useState(value);
5   useEffect(() => {
6     const id = setTimeout(() => setDebounced(value), delay);
7     return () => clearTimeout(id);
8   }, [value, delay]);
9   return debounced;
10 }
```

- useFetch.js

```
# useFetch.js X
campuseats-dashboard > src > features > menu > hooks > # useFetch.js > ...
1 import { useState, useEffect } from "react";
2
3 export function useFetch(url) {
4   const [data, setData] = useState(null);
5   const [isLoading, setIsLoading] = useState(true);
6   const [error, setError] = useState(null);
7
8   useEffect(() => {
9     let active = true;
10    setIsLoading(true);
11    fetch(url)
12      .then((res) => {
13        if (!res.ok) throw new Error("HTTP " + res.status);
14        return res.json();
15      })
16      .then((json) => { if (active) setData(json); })
17      .catch((err) => { if (active) setError(err.message); })
18      .finally(() => { if (active) setIsLoading(false); });
19    return () => { active = false; };
20  }, [url]);
21
22  return { data, isLoading, error };
23 }
```

- The Form (Order.jsx)

```
Order.jsx x
campuseats-dashboard > src > features > menu > pages > Order.jsx > ...
1 import { useState } from "react";
2
3 export default function OrderPage() {
4   const [form, setForm] = useState({ name: "", email: "", qty: 1 });
5   const [errors, setErrors] = useState({});
6   const [done, setDone] = useState(false);
7
8   function validate(v) {
9     const e = {};
10    if (v.name.trim().length < 2) e.name = "Name Too Short!";
11    if (/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v.email)) e.email = "Enter a Valid Email!";
12    if (Number(v.qty) < 1) e.qty = "Quantity must be ≥ 1";
13    return e;
14  }
15
16  function handleChange(e) {
17    setForm({ ...form, [e.target.name]: e.target.value });
18  }
19
20  function handleSubmit(e) {
21    e.preventDefault();
22    const found = validate(form);
23    setErrors(found);
24    if (Object.keys(found).length === 0) setDone(true);
25  }
26
27  if (done) return <p>Thanks, {form.name}! Order Received!</p>;
28
29  return (
30    <form onSubmit={handleSubmit}>
31      <br/>
32      <label for="name">Name: </label><br/>
33      <input id="name" name="name" value={form.name} onChange={handleChange} />
34      {errors.name && <span>{errors.name}</span><br/><br/>
35      <label for="email">Email: </label><br/>
36      <input id="email" name="email" value={form.email} onChange={handleChange} />
37      {errors.email && <span>{errors.email}</span><br/><br/>
38      <label for="qty">Quantity: </label><br/>
39      <input id="qty" name="qty" type="number" value={form.qty} onChange={handleChange} />
40      {errors.qty && <span>{errors.qty}</span><br/><br/>
41      <button type="submit">Place Order</button>
42    </form>
43  );
44 }
```

- The Routes (Inside App.jsx)

```
App.jsx x
campuseats-dashboard > src > App.jsx > ...
1 import { useState } from "react"
2 import { Routes, Route, NavLink } from "react-router-dom"
3 import reactLogo from './assets/react.svg'
4 import viteLogo from './assets/vite.svg'
5 import heroImg from './assets/hero.png'
6 import './App.css'
7 import MenuPage from './features/menu/pages/MenuPage'
8 import DishDetailPage from './features/menu/pages/DishDetailPage'
9 import OrderPage from './features/menu/pages/Order'
10
11 export default function App() {
12   return (
13     <div>
14       <h1>CampusEats</h1>
15       <nav>
16         <NavLink to="/">Menu</NavLink> { " | " }
17         <NavLink to="/order">Place Order</NavLink>
18       </nav>
19       <Routes>
20         <Route path="/" element={<MenuPage />} />
21         <Route path="/dish/:id" element={<DishDetailPage />} />
22         <Route path="/order" element={<OrderPage />} />
23         <Route path="*" element={<p>404 - Page Not Found!</p>} />
24       </Routes>
25     </div>
26   )
27 }
28
```

## Quick Test – Knowledge Check

### Multiple Choice (Choose One)

- Q1. (b) server state
- Q2. (c) TanStack Query
- Q3. (b) an unstable (newly created) prop
- Q4. (c) useCallback
- Q5. (b) only if the API enforces authorization

### Short Answer

Q6.

- In a feature-based folder structure, files are grouped by feature or domain (such as menu, order, or user) instead of by file type. This avoids the file-type (layer-based) anti-pattern, where components, hooks, and pages for the same feature are scattered across different folders, making the project harder to maintain as it grows.

### Code-based

Q7.

```
const [query, setQuery] = useState("");  
<input value={query} onChange = { (e) => setQuery( e.target.value ) } />
```

### Scenario-based

Q8.

- (a) Whether a modal is open,
- **Tool:** useState
  - **Reason:** It is local UI state used only by a single component.
- (b) The product list returned by the API,
- **Tool:** TanStack Query / useFetch
  - **Reason:** It is server state that requires fetching, caching, and loading/error handling.
- (c) The logged-in user shared across the whole app,
- **Tool:** Context API
  - **Reason:** It is shared application-wide state that multiple components need to access.